



```
import A5O
import Closure
set_option autoImplicit false
```

```
/-! # A5O — The Control Plane.
```

```
Imports A5O.lean v4 (SHA-256 0e5ebe3f...5b452) and Closure.lean
(SHA-256 af6b3233...a933) unmodified.
```

A CONTROL PLANE for accountable delegated authority is defined here as a gated system (nothing executes in an area of consequence unless the system decided it for some principal — Closure's `GatedSystem`) that achieves ADA (v4's `achievesADA`: every decision it enforces satisfies O).

Proved below, from that definition alone:

1. every consequential execution is decided by the plane and has a principal satisfying O;
2. under CFunctional, that principal is unique (single governor);
3. any two control planes agree on who governs a shared execution — there is no rival governor;
4. every control plane's decisions lie inside the one canonical O — planes differ only in how much of O they permit, never in its form;
5. revocation halts the act itself, not only the enforcement;
6. boundaries: without the gate there is no plane (Closure), and without CFunctional two governors are possible (v4 quorum case);
7. non-vacuity: a concrete control plane with a real execution exists. -/

section Plane

```
variable {Principal Agent Action Time AoC : Type}
variable (C : Principal → Agent → Action → Time → Prop)
variable (I : Principal → Prop)
variable (M : Agent → Action → Prop)
variable (S : Agent → Action → Prop)
variable (R : Principal → Agent → Action → Time → Prop)
variable (Appointed : Principal → Agent → Action → Time → Prop)
variable (Executes : Agent → Action → Time → Prop)
variable (inAoC : Action → AoC → Prop)
```

```
/-- The control plane: a gate in the execution path that achieves ADA. -/
```

```
structure ControlPlane where
```

```
  gate : GatedSystem (Principal := Principal) (Time := Time) Executes inAoC
```

ada : achievesADA C I M S R Appointed inAoC gate.sys

-- (1) Every consequential execution is decided by the plane, and the deciding principal satisfies O. -/

theorem plane_decides_every_execution

(CP : ControlPlane C I M S R Appointed Executes inAoC)

(A : Agent) (X : Action) (t : Time) (d : AoC)

(hD : inAoC X d) (hE : Executes A X t) :

$\exists P : \text{Principal}, \text{CP.gate.sys.enforces } P \text{ A X t} \wedge \text{O C I M S R Appointed } P \text{ A X t} :=$

let $\langle P, \text{hEnf} \rangle := \text{CP.gate.intercepts } A \text{ X t d hD hE}$

$\langle P, \text{hEnf}, \text{CP.ada } P \text{ A X t d hD hEnf} \rangle$

-- (1') The plane yields v4/Closure accountability over what executes. -/

theorem plane_accountable

(CP : ControlPlane C I M S R Appointed Executes inAoC) :

Accountable C I M S R Appointed Executes inAoC :=

gated_sufficiency C I M S R Appointed Executes inAoC CP.gate CP.ada

-- (2) Single governor: under CFunctional, every consequential execution has exactly one principal for whom the plane decided it. -/

theorem plane_single_governor

(hC : CFunctional C)

(CP : ControlPlane C I M S R Appointed Executes inAoC)

(A : Agent) (X : Action) (t : Time) (d : AoC)

(hD : inAoC X d) (hE : Executes A X t) :

$\exists P : \text{Principal}, \text{CP.gate.sys.enforces } P \text{ A X t} \wedge$

$\forall P' : \text{Principal}, \text{CP.gate.sys.enforces } P' \text{ A X t} \rightarrow P' = P := \text{by}$

obtain $\langle P, \text{hEnf}, \text{hO} \rangle := \text{plane_decides_every_execution } C \text{ I M S R Appointed Executes inAoC CP A X t d hD hE}$

refine $\langle P, \text{hEnf}, \text{fun } P' \text{ hEnf}' => ?_ \rangle$

exact O_unique C I M S R Appointed hC P' P A X t (CP.ada P' A X t d hD hEnf') hO

-- (3) No rival governor: two control planes over the same O, deciding the same consequential act, name the same principal. -/

theorem planes_agree_on_governor

(hC : CFunctional C)

(CP₁ CP₂ : ControlPlane C I M S R Appointed Executes inAoC)

(P₁ P₂ : Principal) (A : Agent) (X : Action) (t : Time) (d : AoC)

(hD : inAoC X d)

(h₁ : CP₁.gate.sys.enforces P₁ A X t) (h₂ : CP₂.gate.sys.enforces P₂ A X t) :

$P_1 = P_2 :=$
 $O_unique\ C\ I\ M\ S\ R\ Appointed\ hC\ P_1\ P_2\ A\ X\ t\ (CP_1.ada\ P_1\ A\ X\ t\ d\ hD\ h_1)\ (CP_2.ada\ P_2\ A\ X\ t\ d\ hD\ h_2)$

/-- (4) One form: every control plane's decisions lie inside the canonical monitor (v4's `CanonicalMonitor`, which enforces exactly O). -/
 theorem plane_within_canonical

(CP : ControlPlane C I M S R Appointed Executes inAoC)
 (P : Principal) (A : Agent) (X : Action) (t : Time) (d : AoC)
 (hD : inAoC X d) (hEnf : CP.gate.sys.enforces P A X t) :
 (CanonicalMonitor C I M S R Appointed).enforces P A X t :=
 CP.ada P A X t d hD hEnf

/-- (5) Revocation halts the act. If the binding at t' names P for (A, X) and P revoked after appointing, then under CFunctional no control plane lets A execute X at t' at all — there is no other principal to decide it. -/
 theorem revocation_halts_execution

[LT Time]
 (Revoke : Principal → Agent → Action → Time → Prop)
 (hP2 : ∀ (P : Principal) (A : Agent) (X : Action) (t t' : Time),
 t < t' → Appointed P A X t → Revoke P A X t' → ¬ R P A X t')
 (hC : CFunctional C)
 (CP : ControlPlane C I M S R Appointed Executes inAoC)
 (P : Principal) (A : Agent) (X : Action) (t t' : Time) (d : AoC)
 (hD : inAoC X d) (hlt : t < t')
 (hApp : Appointed P A X t) (hRev : Revoke P A X t')
 (hBind : C P A X t') :
 ¬ Executes A X t' := by
 intro hE
 obtain ⟨P', hEnf, hO⟩ := plane_decides_every_execution C I M S R Appointed
 Executes inAoC CP A X t' d hD hE
 have hEq : P' = P := hC P' P A X t' hO.1 hBind
 subst hEq
 exact revocation_blocks_enforcement C I M S R Appointed Revoke inAoC hP2
 CP.gate.sys CP.ada P' A X t t' d hD hlt hApp hRev hEnf

end Plane

/-! ## Boundaries -/

/-- (6a) Without the gate there is no plane: the silent system that achieves ADA by enforcing nothing cannot be the gate of any control plane. -/

theorem silent_is_no_plane :

```

  ¬ ∃ CP : ControlPlane cmC cmI cmM cmS cmR cmAppointed cmExecutes
cmInAoC,
    CP.gate.sys = SilentSystem := by
intro ⟨CP, h⟩
exact silent_not_gatable ⟨CP.gate, h⟩

```

/-- (6b) Without CFunctional, a control plane can carry two governors for one act: the v4 quorum instance, gated by the canonical monitor. -/

def quorumPlane :

```

ControlPlane quorumC quorumI quorumM quorumS quorumR quorumAppointed
  (fun (_ : Unit) (_ : Unit) (_ : Unit) => True) (fun (_ : Unit) (_ : Unit) => True) where
gate :=
  { sys := CanonicalMonitor quorumC quorumI quorumM quorumS quorumR
quorumAppointed
  intercepts := fun _ _ _ _ _ =>
    ⟨QuorumPrin.a, quorum_two_governors.1⟩ }
ada := canonical_monitor_achieves_ADA quorumC quorumI quorumM quorumS
quorumR quorumAppointed _

```

theorem quorum_plane_two_governors :

```

quorumPlane.gate.sys.enforces QuorumPrin.a () () () ∧
quorumPlane.gate.sys.enforces QuorumPrin.b () () () ∧
QuorumPrin.a ≠ QuorumPrin.b :=
quorum_two_governors

```

/-! ### Non-vacuity: a concrete control plane with a real execution. -/

```

def wC : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => True
def wI : Unit → Prop := fun _ => True
def wM : Unit → Unit → Prop := fun _ _ => True
def wS : Unit → Unit → Prop := fun _ _ => True
def wR : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => True
def wApp : Unit → Unit → Unit → Unit → Prop := fun _ _ _ _ => True
def wExec : Unit → Unit → Unit → Prop := fun _ _ _ => True
def wAoC : Unit → Unit → Prop := fun _ _ => True

```

def witnessPlane : ControlPlane wC wI wM wS wR wApp wExec wAoC where

```

gate :=
{ sys := CanonicalMonitor wC wI wM wS wR wApp
  intercepts := fun _____ => ⟨(), trivial, trivial, trivial, trivial, trivial, trivial⟩ }
ada := canonical_monitor_achieves_ADA wC wI wM wS wR wApp wAoC

```

```

theorem wC_functional : CFunctional wC := fun _____ => rfl

```

```

/-- (7) The definition is not empty: a control plane exists, an execution
occurs under it, CFunctional holds, and that execution has its one governor. -/

```

```

theorem control_plane_nonvacuous :

```

```

  wExec () () () ∧
  ∃ P : Unit, witnessPlane.gate.sys.enforces P () () () ∧
  ∀ P' : Unit, witnessPlane.gate.sys.enforces P' () () () → P' = P :=
  ⟨trivial, plane_single_governor wC wI wM wS wR wApp wExec wAoC
    wC_functional witnessPlane () () () () trivial trivial⟩

```

```

#print axioms plane_decides_every_execution
#print axioms plane_accountable
#print axioms plane_single_governor
#print axioms planes_agree_on_governor
#print axioms plane_within_canonical
#print axioms revocation_halts_execution
#print axioms silent_is_no_plane
#print axioms quorum_plane_two_governors
#print axioms control_plane_nonvacuous

```